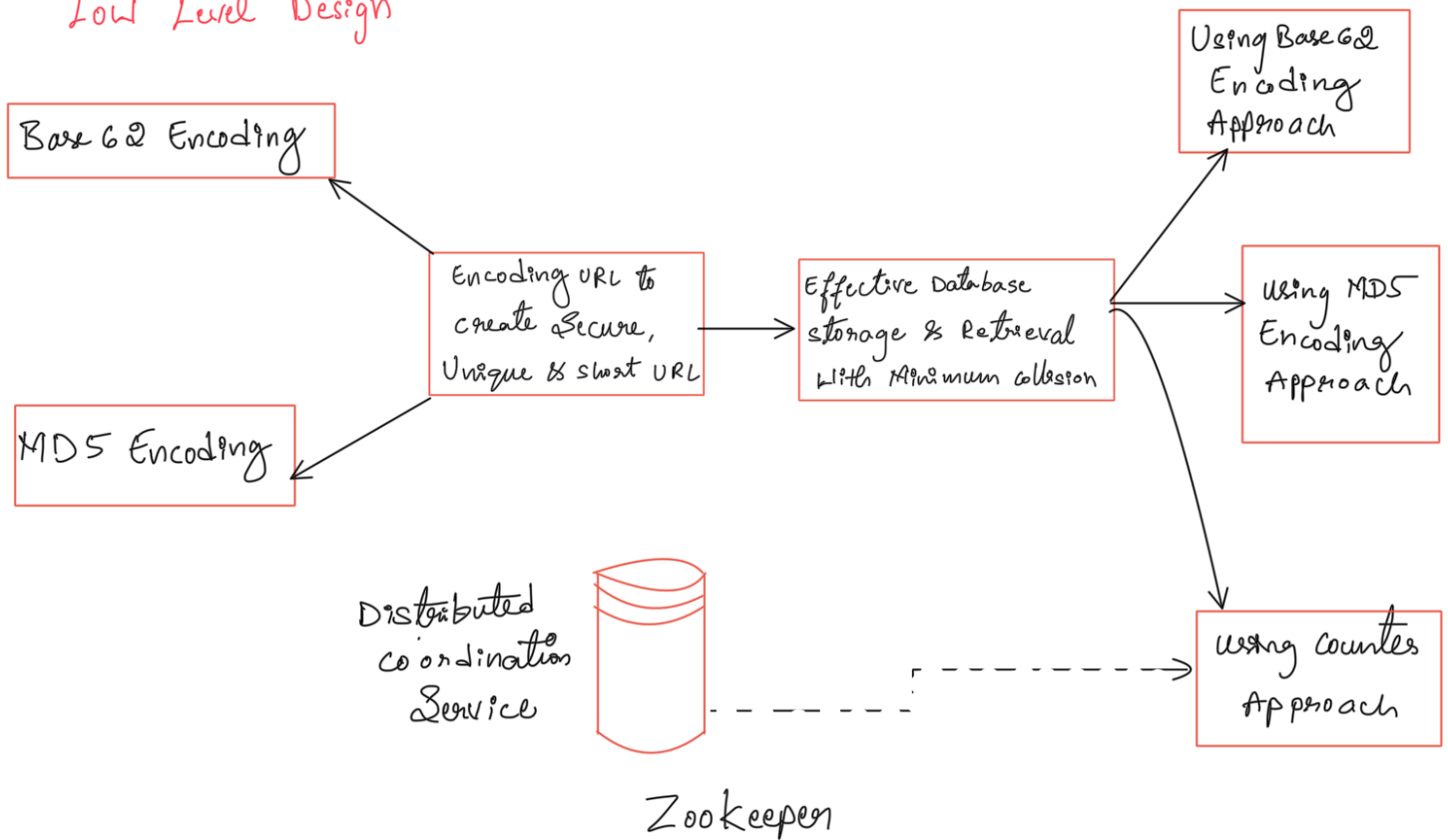


Low Level Design



URL Encoding techniques to create shortened URL

To convert a long URL into a unique short URL we can use Some Hashing techniques like Base62 or MDS.

We will Explore Both Approaches

1. Base62 Encoding

- Base62 Encoding allows us to use the Combination of Characters and numbers which contains A-Z, a-z, 0-9 total (26 + 26 + 10 = 62)
- So for 7 characters short URL, we can serve $62^7 \approx 3500$ billion URLs which is quite enough in comparison to Base10 (Base 10 only contains numbers 0-9 so you will get only 10M combinations)

- We can generate a random Number for the given long URL and convert it to Base62 and use the hash as a Short URL id.

if we use base62 making the assumption that the service is generating 1000 tiny URL's/sec then it will take 110 years to exhaust this 3500 billion combinations.

JAVA Code

```
public String toBase62(int deci) {
    String s =
    "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    String hashStr = "";
    while (deci > 0) {
        hashStr = s.charAt(deci % 62) + hashStr;
        deci /= 62;
    }
    return hashStr;
}
```

62 Chars

8. MD5 Encoding

MD5 also gives Base62 output but the MD5 hash gives a lengthy output which is more than 7 characters

- MD5 hash generates 128-bit long output so out of 128 bits we will take 43 bits to generate a tiny URL of 7 characters.
- MD5 can create a lot of collisions, for two or more or many different long URL inputs we may get the same unique id for a short URL and that could cause data corruption.
- So we need to perform some checks to ensure that this unique id doesn't exist in the database already.

Efficient Database storage & Retrieval of TinyURL

Lets discuss the mapping of a long URL into a short URL in our database

1. Using Base62 Encoding

Assume we generate the tiny URL using base62 encoding then we need to perform the steps given below.

- Tiny URL should be unique so firstly check the existence of this tiny URL in the database (doing `get(tiny)` on DB).
if its already present there for some other long URL then generate a new short URL
- if the short URL isn't present in DB then put the long URL and TinyURL in DB (`put(TinyURL, long URL)`).

This technique work with one server very well but if there will be multiple servers then this technique will create a race condition.

- When multiple servers will work together, there will be a possibility that they all can generate the same Unique ID or the same tiny URL for different long URL's
- Even after checking the database, they will be allowed to insert the same tiny URL's simultaneously in the database and this may end up corrupting the data.

2. Using MD5 Approach

- Encode the long URL using the MD5 approach and take only the first 7 chars to generate tinyURL
- The first 7 characters could be the same for different long URLs

Also check the DB (as we have discussed in technique 1) to verify that TinyURL is not used already

This approach saves some space in the database but how?

- If two users want to generate a tiny URL for the same long URL then the first technique will generate two random numbers and it requires two rows in the database.
- In the second technique, both the longer URL will have the same MD5 so it will have the same first 43 bits.
- This means we will get some deduping and we will end up with saving some space since we only need to store one row instead of two rows in the database.

3. Using Counter Approach

Using a counter is a good decision for a scalable solution because counters always get incremented so we can get a new value for every new request.

Single Server Approach:

- A single host or server (say database) will be responsible for maintaining the counter.
- When the worker host receives a request it talks to the counter host, which returns a unique number and increments the counter.
When the next request comes the counter host again returns the unique number and this goes on.
- Every worker host gets a unique number which is used to generate TinyURL.